

Screen-Space Ray Tracing: Implementation Report

Real-Time Rendering, MSc Computer Science, Trinity College Dublin

Gaurav Naudiyal

10th April 2026

Abstract

This report covers the design and implementation of a real-time Screen-Space Ray Tracing (SSRT) renderer built using OpenGL 3.3 Core Profile and GLSL. The system uses a three-attachment G-Buffer for deferred shading and applies view-space ray marching with binary search refinement to produce dynamic reflections. The approach is loosely inspired by McGuire and Mara (2014) but differs significantly in traversal method. An ImGui overlay provides live performance monitoring and reflections can be toggled at runtime for comparison. The report discusses the pipeline, key implementation decisions, performance results, and limitations.

1 Background: Summary of the Technical Paper

The work draws from McGuire and Mara’s 2014 paper *Efficient GPU Screen-Space Ray Tracing* [1], which proposes a method for producing dynamic reflections using only the data already available in a deferred shading G-Buffer. The key insight is that the depth buffer contains enough geometric information to approximate reflections without tracing rays through actual scene geometry.

1.1 What the Paper Proposes

The paper’s core contribution is adapting the perspective-correct Digital Differential Analyser (DDA) line rasterisation algorithm to 3D ray traversal against a depth buffer. Rather than stepping forward in 3D view space at equal intervals, the ray is projected into 2D screen space and marched pixel-by-pixel along that projected line [1]. This guarantees each screen pixel along the ray is sampled at most once, avoiding both oversampling and undersampling regardless of the ray’s angle. Perspective correction is maintained throughout so reconstructed 3D positions remain accu-

rate at oblique angles.

On top of the DDA traversal, the paper proposes a **Hierarchical Z-Buffer (Hi-Z)** as an acceleration structure. A mip-chain of the depth buffer is built each frame where each coarser level stores the minimum depth over a 2x2 region. The ray can then skip large sections of empty depth space by querying coarse mip levels, only stepping down to finer levels when a potential hit is detected [2]. This is what makes the paper’s method genuinely scalable for long-distance rays.

1.2 What Was Implemented Instead

This implementation uses a simpler **uniform linear ray march in view space** rather than the DDA traversal or Hi-Z acceleration. Building a Hi-Z mip-chain requires generating a depth pyramid each frame via compute shaders or multi-pass blits, which was beyond the scope of this project. The uniform march is easier to debug and still demonstrates the core deferred reflections pipeline, though it comes with accuracy trade-offs at oblique angles. Table 1 summarises the key differences.

Table 1: Paper vs. implementation comparison

Aspect	McGuire & Mara (2014)	This Implementation
Traversal method	Perspective-correct 2D DDA in screen space	Uniform step march in 3D view space
Sampling guarantee	Each pixel sampled at most once	Can over/undersample at oblique angles
Acceleration	Hi-Z depth mip-chain	None; bounded by <code>MAX_STEPS = 64</code>
Hit refinement	Binary search	Binary search (8 steps, adopted from paper)

2 Implementation Details

2.1 Two-Pass Deferred Pipeline

The renderer uses two passes. In the first pass all scene geometry is drawn into a Framebuffer Object (FBO) with three colour attachments forming the G-Buffer. In the second pass a fullscreen quad reads from those textures to compute lighting and reflections. This means geometry transformation happens once per frame and the ray march runs once per screen pixel rather than per geometry fragment.

The G-Buffer stores everything in view space, which simplifies the reflection computation since the view direction for any fragment is simply `normalize(fragPos)`. Normals use the inverse-

transpose of the model-view matrix for correct handling under non-uniform scaling. Reflectivity is packed into the alpha channel of the albedo texture to avoid a fourth attachment.

Table 2: G-Buffer layout

Attachment	Format	Contents
gPosition	RGB16F	View-space position
gNormal	RGB16F	View-space normal
gAlbedoSpec	RGBA8	Albedo (RGB) + Reflectivity (A)

2.2 Ray March and Binary Search

`traceRay()` steps forward by `STEP_SIZE = 0.2` view-space units per iteration up to `MAX_STEPS = 64` times. At each step the position is projected to screen UV and the G-Buffer depth is sampled. A hit is registered when the ray depth falls behind the scene depth within a `THICKNESS = 0.5` tolerance. A coarse hit triggers `binarySearch()`, which runs 8 refinement steps by halving the step vector and adjusting direction based on overshoot or undershoot. This refinement, taken from the paper, visibly improves reflection sharpness along geometry edges.

Listing 1: Hit detection and binary search entry (`ssrt.frag`)

```
if (rayPos.z < sceneZ && rayPos.z > sceneZ - THICKNESS) {
    hitUV = binarySearch(rayPos, step);
    return true;
}
```

A `JITTER = 0.25` offset at ray origin reduces temporal banding, and a smoothstep edge fade ramps reflections to zero over the outer 10% of the screen to hide missing-data artifacts at boundaries.

2.3 Scene

The scene has a reflective floor (reflectivity 0.9), a semi-reflective back wall (0.4), four static cubes of varying colours and reflectivities, and one yellow cube that orbits the scene centre each frame using a time-driven sine/cosine offset. The animated cube is useful for validating that reflections update correctly for fully dynamic objects.

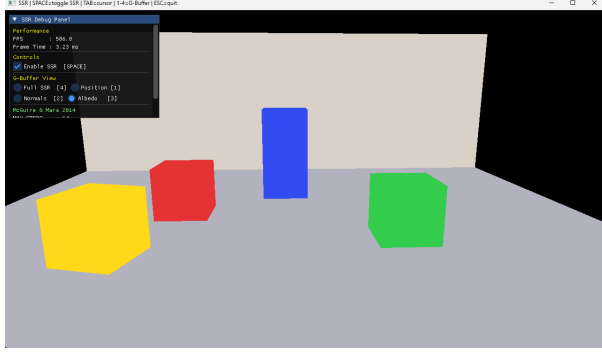


Figure 1: Albedo G-Buffer (key: 3)

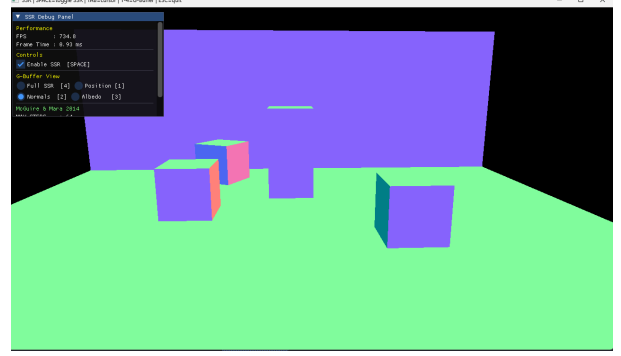


Figure 2: Normal G-Buffer (key: 2)

3 Results and Evaluation

3.1 Performance at 1280x720

Table 3: Measured frame performance

Condition	FPS	Frame Time (ms)
SSR Enabled	196.7	4.68ms
SSR Disabled	940.1	1.80ms

The dominant cost is the 64-iteration march running for every fragment with reflectivity above 0.01. At worst case with the full floor filling the viewport, a large number of pixels run the entire loop. Reducing MAX_STEPS from 64 to 32 halved SSR frame time with a visible quality loss on distant reflections. The binary search adds 8 extra lookups only on successful hits, so its overhead is minor relative to the quality gain.

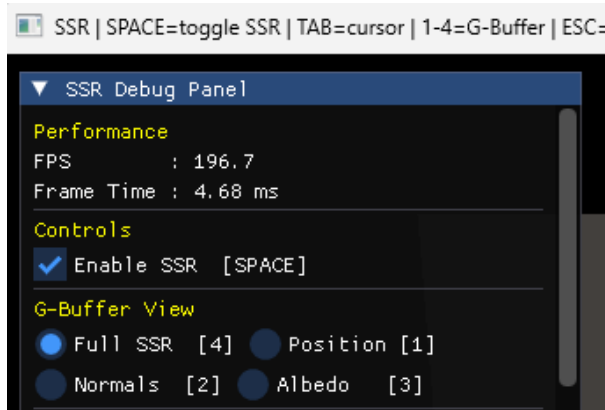


Figure 3: ImGui panel, SSR ON

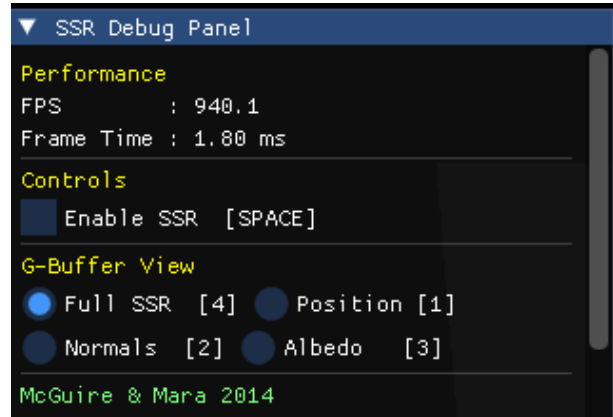


Figure 4: ImGui panel, SSR OFF

3.2 Visual Quality

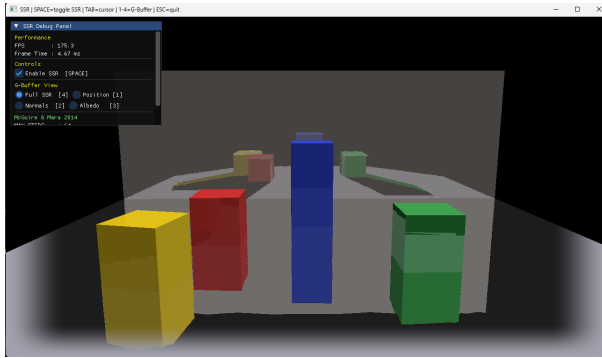


Figure 5: SSR enabled

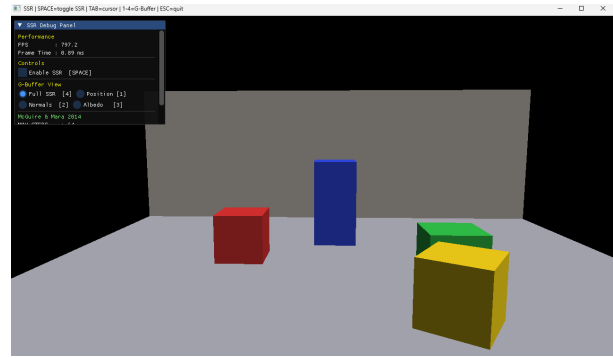


Figure 6: SSR disabled (diffuse only)

Reflections work well at close range and update correctly in real time for the animated cube. The binary search refinement reduces stairstepping at geometry edges. Rays at shallow oblique angles produce slightly offset reflections because the fixed view-space step size does not adapt to perspective foreshortening the way the paper’s DDA approach does. This is the most noticeable visual limitation of choosing uniform stepping over the paper’s method.

3.3 Resource Usage

The two RGB16F G-Buffer textures use roughly 5.5 MB each and the RGBA8 albedo texture around 3.7 MB, giving a total G-Buffer footprint of approximately 15 to 16 MB at 1280x720. No compute shaders or additional passes are required beyond the two-pass structure.

4 Improvements and Limitations

Hi-Z traversal. Building the depth mip-chain described in the paper would allow rays to skip empty depth space and significantly improve quality for long-distance reflections without increasing MAX_STEPS [2].

DDA screen-space marching. Replacing the uniform view-space step with the paper’s 2D perspective-correct DDA would fix oblique-angle quality issues and match the paper’s sampling guarantee.

Temporal reprojection. Adding a reprojection buffer blended with motion vectors would stabilise the reflections during fast camera movement, which currently causes minor flickering.

Environment map fallback. When a ray misses, the reflection defaults to a near-black constant. Sampling a pre-baked environment cube map for missed rays would make off-screen reflections plausible [3].

Roughness support. All reflections are currently sharp regardless of material. Stochastically jittering the reflection direction based on a roughness value would allow diffuse and glossy surfaces.

Half-resolution pass. Running the SSR pass at half resolution with a bilateral upscale would roughly halve the fragment shader cost on reflective surfaces.

References

- [1] McGuire, M. and Mara, M. (2014). *Efficient GPU Screen-Space Ray Tracing*. JCGT, 3(4), 73–85. <https://jcggt.org/published/0003/04/04/>
- [2] Pettineo, M. (2022). *Notes on Screenspace Reflections with FidelityFX SSSR*. <https://interplayoflight.wordpress.com/2022/09/28/notes-on-screenspace-reflections-with-fidelityfx-sssr/>
- [3] Pham, A. (2019). *Adventures in Deferred Rendering: Screen Space Reflections*. <https://andrewphamdotblog.wordpress.com/2019/07/31/screen-space-reflections/>
- [4] Khronos Group (2024). *OpenGL 3.3 Core Profile Specification*. <https://www.khronos.org/opengl/>
- [5] Cornut, O. (2024). *Dear ImGui*. <https://github.com/ocornut/imgui>
- [6] GLFW Contributors (2024). *GLFW*. <https://www.glfw.org/>
- [7] G-Truc Creation (2024). *OpenGL Mathematics (GLM)*. <https://github.com/g-truc/glm>
- [8] Dav1d (2024). *GLAD OpenGL Loader*. <https://github.com/Dav1dde/glad>